Justificativa da divisão dos microsserviços e tecnologias utilizadas

A solução Officyna iniciou como uma aplicação monolítica, na qual as responsabilidades de atendimento, ordem de serviço, clientes, veículos, catálogo de mão de obra, estoque, execução, faturamento, autenticação e monitoramento coexistiam em uma única base de código e compartilhavam a mesma persistência. Embora esse modelo tenha sido adequado para validar o MVP, ele passou a apresentar alto acoplamento entre domínios que possuem ritmos de mudança, requisitos de segurança e necessidades de escala distintos.

A evolução para microsserviços foi realizada com base na separação por contexto de negócio (*bounded contexts*), buscando garantir que cada serviço seja responsável por uma capacidade de negócio específica, mantenha seus próprios dados e possa ser implantado e escalado de forma independente.

Divisão dos serviços

Serviço	Responsabilidade	Justificativa
`officyna-os-service`	Gestão de clientes, veículos e ciclo de vida da Ordem de Serviço.	A Ordem de Serviço é o núcleo do negócio da oficina. O serviço concentra a abertura, consulta e transições de status da OS, preservando a consistência do fluxo principal de atendimento. Itens de mão de obra e peças são tratados como referências de domínios externos, reduzindo o acoplamento.
`officyna-execution-production`	Diagnóstico, execução da OS, catálogo de mão de obra, peças, estoque e monitoramento operacional.	A operação da oficina possui regras próprias, como reserva e consumo de estoque, início e término de mão de obra, acompanhamento do diagnóstico e cálculo de tempos de execução. Esse domínio tende a sofrer mudanças frequentes e exige escala independente em períodos de maior operação, justificando sua separação do núcleo da OS.
`officyna-billing-service`	Geração de orçamento, cálculo de impostos, checkout, pagamentos e conciliação.	O domínio financeiro exige maior isolamento por envolver integrações externas, como Mercado Pago, e dados sensíveis de pagamento. A separação reduz o impacto de indisponibilidades externas sobre a abertura e a execução das ordens de serviço, além de

		permitir evolução independente das regras fiscais e de cobrança.
`officyna-lambda`	Autenticação de clientes por CPF e emissão de JWT.	A autenticação por CPF é uma operação pontual e sob demanda. A implementação serverless permite escalar conforme o número de acessos, reduz custo quando não há requisições e centraliza a emissão de tokens para os demais serviços.
`officyna-users-service`	Gestão de usuários internos, como administradores, atendentes e mecânicos.	A gestão de identidade de colaboradores é um contexto diferente da gestão de clientes e da operação da OS. A separação permite futuramente centralizar perfis, permissões e políticas de acesso sem acoplar essas regras ao fluxo operacional.
`officyna-infra-db`	Provisionamento do banco e recursos de rede.	A infraestrutura do banco foi separada do código de negócio para aplicar Infraestrutura como Código, permitir versionamento, auditoria e implantação independente dos recursos de dados.
`officyna-infra-k8s`	Provisionamento do cluster Kubernetes e recursos de execução.	O cluster, nós, permissões IAM e rede são responsabilidades de plataforma. Mantê-los em repositório próprio evita que mudanças na aplicação alterem inadvertidamente a infraestrutura compartilhada.

Comunicação entre serviços

A arquitetura utiliza comunicação híbrida. Operações que exigem resposta imediata utilizam APIs REST, como o encaminhamento de alterações da execução para a Ordem de Serviço e a criação de checkout no serviço financeiro. Já eventos de negócio que não exigem resposta síncrona utilizam mensageria com Amazon SQS e SNS.

Essa escolha reduz o acoplamento temporal entre os serviços. Por exemplo, a geração de orçamento, as notificações de pagamento e os eventos de início ou término de execução podem ser processados de forma assíncrona. O uso de `correlationId` nas mensagens permite rastrear uma mesma OS ao longo de diferentes serviços e apoiar a observabilidade do fluxo distribuído.

As filas também contribuem para resiliência, pois absorvem picos de processamento e permitem reprocessamento controlado. No fluxo de execução, há filas de retentativa e DLQ (*Dead Letter Queue*) para tratar falhas sem perda imediata de mensagens.

Persistência e autonomia dos dados

Cada microsserviço mantém a responsabilidade sobre seus próprios dados. O `officyna-os-service` utiliza MongoDB/Amazon DocumentDB para clientes, veículos e ordens de serviço. O `officyna-execution-production` mantém bancos lógicos próprios para catálogo de mão de obra, peças, estoque, monitoramento e snapshots de execução. O `officyna-billing-service` utiliza PostgreSQL com Flyway para armazenar orçamentos e pagamentos.

A adoção de MongoDB/DocumentDB é adequada aos domínios de OS e execução porque permite flexibilidade para documentos com histórico de status, dados de diagnóstico e snapshots operacionais. Já o PostgreSQL no faturamento é apropriado para entidades financeiras, que demandam consistência transacional, integridade relacional e migrações de esquema controladas.

Essa segregação evita acesso direto de um serviço ao banco de outro. Por exemplo, o serviço de execução não acessa a base da OS: ele recebe eventos, mantém somente seu snapshot operacional e utiliza APIs ou mensagens para atualizar o serviço responsável pela Ordem de Serviço.

Tecnologias utilizadas e justificativas

Tecnologia	Uso na solução	Justificativa
Java 21 e Spring Boot	Serviços de OS, execução e faturamento.	Oferecem maturidade para APIs REST, validação, segurança, integração com dados, testes e desenvolvimento de regras de negócio.
Node.js 20 e TypeScript	Função de autenticação serverless.	Adequados para funções leves, com inicialização rápida e tipagem estática para maior segurança no desenvolvimento.
AWS Lambda	Validação de CPF e emissão de JWT.	Permite execução sob demanda, escalabilidade automática e menor custo operacional para uma função acionada por requisição.
MongoDB / Amazon DocumentDB	Dados de OS, clientes, veículos, execução, estoque e catálogo operacional.	Suportam estrutura documental e evolução flexível do modelo de dados. O DocumentDB fornece operação gerenciada na AWS e compatibilidade com a API MongoDB.
PostgreSQL e Flyway	Orçamentos e pagamentos.	PostgreSQL oferece transações e integridade para o domínio financeiro; Flyway versiona e automatiza as migrações do banco.

Amazon SQS e SNS	Eventos assíncronos entre serviços.	SQS desacopla produtores e consumidores; SNS permite publicar eventos para múltiplos interessados, como pagamento confirmado para OS e execução.
Mercado Pago	Checkout e confirmação de pagamentos.	Permite suportar Pix e cartão sem que a aplicação manipule dados brutos de cartão, reduzindo exposição de informações sensíveis.
JWT e Spring Security	Autenticação e autorização.	JWT permite que os serviços validem credenciais de forma distribuída, enquanto as roles controlam o acesso às operações protegidas.
Kong API Gateway	Porta de entrada das APIs.	Centraliza roteamento, exposição de endpoints e aplicação de políticas de segurança, evitando que cada serviço seja exposto diretamente.
Docker e Docker Compose	Execução local e empacotamento das aplicações.	Padronizam os ambientes de desenvolvimento, teste e execução, reduzindo diferenças entre máquinas.
Kubernetes / Amazon EKS	Orquestração dos serviços em nuvem.	Permite execução em contêineres, replicação, recuperação de falhas e escalabilidade horizontal por meio de HPA.
Terraform	Provisionamento de infraestrutura.	Garante infraestrutura versionada, reproduzível e auditável para VPC, DocumentDB, EKS, permissões e recursos de rede.
GitHub Actions	Integração e entrega contínuas.	Automatiza validações, testes, geração de imagens e implantação, reforçando a governança dos repositórios.
OpenAPI/Swagger, Actuator, logs JSON e New Relic	Documentação e observabilidade.	Facilitam o consumo das APIs, monitoramento de saúde, rastreabilidade por `correlationId` e acompanhamento de métricas operacionais.
JaCoCo, SonarQube e OWASP Dependency Check	Qualidade e segurança de código.	Apoiam cobertura de testes, análise estática e identificação de vulnerabilidades em dependências.

Benefícios da arquitetura adotada

A divisão realizada reduz o impacto de mudanças locais, pois alterações em pagamento, estoque ou autenticação não exigem uma nova implantação do núcleo de Ordem de Serviço. Também permite escalar cada capacidade de acordo com sua demanda: autenticação pode escalar com Lambda, execução pode aumentar réplicas no Kubernetes em períodos de maior movimento e faturamento pode ser protegido de falhas ou lentidão de provedores externos.

Além disso, a separação fortalece a segurança e a governança. Dados financeiros ficam isolados no serviço de faturamento, a autenticação é centralizada na Lambda e a infraestrutura possui repositórios próprios, com Terraform e pipelines independentes. Assim, a arquitetura deixa de ser uma aplicação única e fortemente acoplada para se tornar um ecossistema distribuído, orientado a domínios de negócio, escalável, observável e preparado para evolução contínua.